AI ASSISTED CODING
ASSESMENT-12.1

Name: P.Revanth
HT No: 2303A52421
Batch: 32

Task Description #1 (Sorting – Merge Sort Implementation)
• Task: Use AI to generate a Python program that implements the Merge Sort algorithm.
• Instructions:
o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.
o Ask AI to include time complexity and space complexity in the function docstring.
o Verify the generated code with test cases.
• Expected Output:
o A functional Python script implementing Merge Sort with proper documentation.

Code:

```
# Create a function merge_sort(arr)
# Include Time Complexity and Space Complexity in the Docstring
# Include Some Test Cases To Verify
def merge_sort(arr):
    """
    Sorts an array using the merge sort algorithm.

    Time Complexity: O(n log n)
    Space Complexity: O(n)

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
```

```
        return arr

    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])

    return merge(left_half, right_half)
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.
    Parameters:
    left (list): The first sorted list.
    right (list): The second sorted list.
    Returns:
    list: A merged and sorted list.
    """
    merged = []
    left_index = right_index = 0

    while left_index < len(left) and right_index < len(right):
        if left[left_index] < right[right_index]:
            merged.append(left[left_index])
            left_index += 1
        else:
            merged.append(right[right_index])
            right_index += 1

    merged.extend(left[left_index:])
    merged.extend(right[right_index:])

    return merged
# Test Cases
if __name__== "__main__":
    test_cases = [
        [38, 27, 43, 3, 9, 82, 10],
        [5, 2, 9, 1, 5, 6],
        [12, 11, 13, 5, 6, 7],
        [],
        [1],
        [2, 1]
    ]

    for i, test in enumerate(test_cases):
```

```
        print(f"Test Case {i + 1}: {merge_sort(test)}")
```

Output:
Test Case 1: [3, 9, 10, 27, 38, 43, 82]
Test Case 2: [1, 2, 5, 5, 6, 9]
Test Case 3: [5, 6, 7, 11, 12, 13]
Test Case 4: []
Test Case 5: [1]
Test Case 6: [1, 2]

Justification:
Merge Sort was chosen because it guarantees O(n log n) time in best, average, and worst cases, making it reliable for larger unsorted datasets. Its divide-and-conquer design is stable and predictable, though it needs extra memory of O(n).

Task Description #2 (Searching – Binary Search with AI Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

Code:

```
# Create a function binary_search(arr, target)
# Return index if found, else return -1
# Use iterative or recursive method
# Add docstring with best, average, worst complexity
# Include multiple test cases
def binary_search(arr, target):
    """
    Perform binary search on a sorted array.

    Parameters:
    arr (list): A sorted list of elements.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.

    Complexity:
    Best Case: O(1) - when the target is at the middle of the array.
    Average Case: O(log n) - when the target is found after several
iterations.
    Worst Case: O(log n) - when the target is not found or is at the
end of the array.
    """
```

```
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = left + (right - left) // 2

        # Check if the target is present at mid
        if arr[mid] == target:
            return mid
        # If target is greater, ignore the left half
        elif arr[mid] < target:
            left = mid + 1
        # If target is smaller, ignore the right half
        else:
            right = mid - 1

    # Target was not found in the array
    return -1
# Test cases
if __name__== "__main__":
    arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
    print(binary_search(arr, 5))  # Output: 4
    print(binary_search(arr, 1))  # Output: 0
    print(binary_search(arr, 10)) # Output: 9
    print(binary_search(arr, 11)) # Output: -1
    print(binary_search(arr, -1)) # Output: -1
    print(binary_search(arr, 6))  # Output: 5
```

Output:
4
0
9
-1
-1
5

Justification:
Binary Search is optimal for sorted lists because it reduces the search space by half each step, giving O(logn) average/worst-case time. It is much faster than linear search for large data, but requires the input list to be sorted first.

Task Description #3 (Real-Time Application – Inventory Management System)

• Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

• Task:

o Use AI to suggest the most efficient search and sort algorithms for this use case.

o Implement the recommended algorithms in Python.

o Justify the choice based on dataset size, update frequency, and performance requirements.

• Expected Output:

o A table mapping operation → recommended algorithm → justification.

o Working Python functions for searching and sorting the inventory.

Code:

```
"""Task Description #3 - Inventory Management System.

Scenario:
- Large inventory (thousands of products)
- Fast product lookup by ID or name
- Sorting by price or quantity for stock analysis

This script provides:
1. Operation -> Algorithm -> Justification table
2. Working Python functions for search and sort
3. Reasoning based on dataset size, update frequency, and performance
"""

from dataclasses import dataclass
from typing import Dict, List
```

```python
@dataclass(frozen=True)
class Product:
    product_id: int
    name: str
    price: float
    quantity: int


RECOMMENDATION_TABLE = [
    {
        "operation": "Search by product ID",
        "algorithm": "Hash index (Python dict)",
        "justification": (
            "Direct key lookup gives average O(1) time, which is best
for high-frequency"
            " real-time ID searches on large inventories."
        ),
    },
    {
        "operation": "Search by product name",
        "algorithm": "Normalized hash index (name -> list[Product])",
        "justification": (
            "Case-insensitive average O(1) lookup. Mapping names to
lists handles duplicate"
            " names while keeping query latency low."
        ),
    },
    {
        "operation": "Sort by price",
        "algorithm": "Timsort (Python sorted)",
        "justification": (
            "O(n log n) worst-case, stable, and highly optimized in
CPython. Works very well"
            " for recurring analytics views across thousands of rows."
        ),
    },
    {
        "operation": "Sort by quantity",
        "algorithm": "Timsort (Python sorted)",
        "justification": (
            "Same efficiency and stability benefits as price sorting;
robust for moderate"
            " update rates and repeated reporting queries."
```

```
        ),
    },
]


class Inventory:
    def __init__(self, products: List[Product]) -> None:
        self.products = products
        self._id_index: Dict[int, Product] = {}
        self._name_index: Dict[str, List[Product]] = {}
        self._rebuild_indexes()

    def _rebuild_indexes(self) -> None:
        self._id_index = {p.product_id: p for p in self.products}
        self._name_index = {}
        for product in self.products:
            key = product.name.strip().lower()
            self._name_index.setdefault(key, []).append(product)

    def add_product(self, product: Product) -> None:
        """Insert/update one product and keep indexes in sync in O(1)
average."""
        existing = self._id_index.get(product.product_id)
        if existing is not None:
            self.products.remove(existing)
        self.products.append(product)
        self._id_index[product.product_id] = product
        key = product.name.strip().lower()
        self._name_index.setdefault(key, []).append(product)

    def search_by_id(self, product_id: int) -> Product | None:
        return self._id_index.get(product_id)

    def search_by_name(self, name: str) -> List[Product]:
        return self._name_index.get(name.strip().lower(), [])

    def sort_by_price(self, descending: bool = False) -> List[Product]:
        return sorted(self.products, key=lambda p: p.price,
reverse=descending)

    def sort_by_quantity(self, descending: bool = False) ->
List[Product]:
```

```
        return sorted(self.products, key=lambda p: p.quantity,
reverse=descending)


def print_recommendation_table() -> None:
    print("Operation | Recommended Algorithm | Justification")
    print("-" * 120)
    for row in RECOMMENDATION_TABLE:
        print(f"{row['operation']} | {row['algorithm']} |
{row['justification']}")


def print_algorithm_rationale() -> None:
    print("\nWhy these choices fit this scenario:")
    print("1. Dataset size (thousands of products):")
    print("   - Hash indexes avoid O(n) scans for each search
request.")
    print("2. Update frequency (typically moderate in retail
systems):")
    print("   - Index maintenance per update is small, so fast reads
are preserved.")
    print("3. Performance requirement (real-time staff usage):")
    print("   - ID/name lookups are average O(1), giving responsive
queries.")
    print("   - Sorting is O(n log n), suitable for reporting and stock
analysis views.")


def build_sample_inventory() -> Inventory:
    sample_products = [
        Product(1001, "Laptop", 74999.0, 12),
        Product(1002, "Mouse", 899.0, 180),
        Product(1003, "Keyboard", 1499.0, 120),
        Product(1004, "Monitor", 12499.0, 45),
        Product(1005, "Headphones", 2499.0, 90),
        Product(1006, "Mouse", 1099.0, 70),
    ]
    return Inventory(sample_products)


def run_demo() -> None:
    inventory = build_sample_inventory()
```

```
    print("\nSearch by ID (1004):")
    print(inventory.search_by_id(1004))

    print("\nSearch by Name ('mouse'):")
    print(inventory.search_by_name("mouse"))

    print("\nSort by Price (ascending):")
    print(inventory.sort_by_price())

    print("\nSort by Quantity (descending):")
    print(inventory.sort_by_quantity(descending=True))


def run_tests() -> None:
    inventory = build_sample_inventory()

    # Search tests
    assert inventory.search_by_id(1001) == Product(1001, "Laptop",
74999.0, 12)
    assert inventory.search_by_id(9999) is None

    by_name = inventory.search_by_name("MOUSE")
    assert len(by_name) == 2
    assert all(product.name == "Mouse" for product in by_name)

    # Sort tests
    price_ids = [p.product_id for p in inventory.sort_by_price()]
    assert price_ids == [1002, 1006, 1003, 1005, 1004, 1001]

    quantity_ids_desc = [p.product_id for p in
inventory.sort_by_quantity(descending=True)]
    assert quantity_ids_desc == [1002, 1003, 1005, 1006, 1004, 1001]

    print("\nAll tests passed.")


def main() -> None:
    print("Inventory Management: Efficient Search and Sort")
    print_recommendation_table()
    print_algorithm_rationale()
    run_demo()
    run_tests()
```

```python
if __name__ == "__main__":
    main()
```

Output:
Inventory Management: Efficient Search and Sort
Operation | Recommended Algorithm | Justification

Search by product ID | Hash index (Python dict) | Direct key lookup gives average O(1) time, which is best for high-frequency real-time ID searches on large inventories.
Search by product name | Normalized hash index (name -> list[Product]) | Case-insensitive average O(1) lookup. Mapping names to lists handles duplicate names while keeping query latency low.
Sort by price | Timsort (Python sorted) | O(n log n) worst-case, stable, and highly optimized in CPython. Works very well for recurring analytics views across thousands of rows.
Sort by quantity | Timsort (Python sorted) | Same efficiency and stability benefits as price sorting; robust for moderate update rates and repeated reporting queries.

Why these choices fit this scenario:
1. Dataset size (thousands of products):
   - Hash indexes avoid O(n) scans for each search request.
2. Update frequency (typically moderate in retail systems):
   - Index maintenance per update is small, so fast reads are preserved.
3. Performance requirement (real-time staff usage):
   - ID/name lookups are average O(1), giving responsive queries.
   - Sorting is O(n log n), suitable for reporting and stock analysis views.

Search by ID (1004):

Product(product_id=1004, name='Monitor', price=12499.0, quantity=45)

Search by Name ('mouse'):
[Product(product_id=1002, name='Mouse', price=899.0, quantity=180), Product(product_id=1006, name='Mouse', price=1099.0, quantity=70)]

Sort by Price (ascending):
[Product(product_id=1002, name='Mouse', price=899.0, quantity=180), Product(product_id=1006, name='Mouse', price=1099.0, quantity=70), Product(product_id=1003, name='Keyboard', price=1499.0, quantity=120), Product(product_id=1005, name='Headphones', price=2499.0, quantity=90), Product(product_id=1004, name='Monitor', price=12499.0, quantity=45), Product(product_id=1001, name='Laptop', price=74999.0, quantity=12)]

Sort by Quantity (descending):
[Product(product_id=1002, name='Mouse', price=899.0, quantity=180), Product(product_id=1003, name='Keyboard', price=1499.0, quantity=120), Product(product_id=1005, name='Headphones', price=2499.0, quantity=90), Product(product_id=1006, name='Mouse', price=1099.0, quantity=70), Product(product_id=1004, name='Monitor', price=12499.0, quantity=45), Product(product_id=1001, name='Laptop', price=74999.0, quantity=12)]

All tests passed.

Justification:
For product lookup, hash indexing (dictionary) was used for ID/name access because it gives average O(1) search, ideal for real-time queries on thousands of records.For price/quantity analysis, Python's Timsort O(nlogn)) was used since it is stable, efficient, and practical for repeated reporting views.

Task description #4: Smart Hospital Patient Management System
A hospital maintains records of thousands of patients with details such as patient ID, name, severity level, admission date, and bill amount. Doctors and staff need to:
1. Quickly search patient records using patient ID or name.
2. Sort patients based on severity level or bill amount for prioritization and billing.
Student Task
• Use AI to recommend suitable searching and sorting algorithms.
• Justify the selected algorithms in terms of efficiency and suitability.
• Implement the recommended algorithms in Python.

Code:

```
"""Task #4: Smart Hospital Patient Management System.

Implements:
1. Fast patient search by patient_id or name
2. Sorting by severity_level and bill_amount
3. Algorithm recommendation table with justification
"""

from dataclasses import dataclass
from datetime import date
from typing import Dict, List



@dataclass(frozen=True)
```

```
class Patient:
    patient_id: int
    name: str
    severity_level: int
    admission_date: date
    bill_amount: float


ALGORITHM_TABLE = [
    {
        "operation": "Search by patient ID",
        "algorithm": "Hash index (dict)",
        "justification": (
            "Average O(1) lookup is ideal for frequent real-time access
in large records."
        ),
    },
    {
        "operation": "Search by patient name",
        "algorithm": "Normalized hash index (name -> list[Patient])",
        "justification": (
            "Average O(1) lookup with case-insensitive matching;
handles duplicate names."
        ),
    },
    {
        "operation": "Sort by severity level",
        "algorithm": "Timsort (Python sorted)",
        "justification": (
            "Stable O(n log n) worst-case performance, optimized in
Python for practical data."
        ),
    },
    {
        "operation": "Sort by bill amount",
        "algorithm": "Timsort (Python sorted)",
        "justification": (
            "Efficient and stable for billing analysis and report
generation on large datasets."
        ),
    },
]
```

```
class PatientManagementSystem:
    def __init__(self, patients: List[Patient]) -> None:
        self.patients = list(patients)
        self._id_index: Dict[int, Patient] = {}
        self._name_index: Dict[str, List[Patient]] = {}
        self._rebuild_indexes()

    def _rebuild_indexes(self) -> None:
        self._id_index = {patient.patient_id: patient for patient in
self.patients}
        self._name_index = {}
        for patient in self.patients:
            key = patient.name.strip().lower()
            self._name_index.setdefault(key, []).append(patient)

    def add_or_update_patient(self, patient: Patient) -> None:
        """Insert or update one patient while keeping search indexes
synchronized."""
        old = self._id_index.get(patient.patient_id)
        if old is not None:
            self.patients.remove(old)
        self.patients.append(patient)
        self._rebuild_indexes()

    def search_by_patient_id(self, patient_id: int) -> Patient | None:
        return self._id_index.get(patient_id)

    def search_by_name(self, name: str) -> List[Patient]:
        return self._name_index.get(name.strip().lower(), [])

    def sort_by_severity(self, descending: bool = True) ->
List[Patient]:
        """Default descending: higher severity first for clinical
prioritization."""
        return sorted(self.patients, key=lambda patient:
patient.severity_level, reverse=descending)

    def sort_by_bill_amount(self, descending: bool = True) ->
List[Patient]:
        """Default descending: highest bills first for financial
review."""
```

```
        return sorted(self.patients, key=lambda patient:
patient.bill_amount, reverse=descending)


def print_algorithm_table() -> None:
    print("Operation | Recommended Algorithm | Justification")
    print("-" * 115)
    for row in ALGORITHM_TABLE:
        print(f"{row['operation']} | {row['algorithm']} |
{row['justification']}")


def print_justification_notes() -> None:
    print("\nSuitability Notes")
    print("1. Efficiency for large datasets (thousands of records):")
    print("   - Hash indexes avoid linear scans during common search
operations.")
    print("2. Real-time hospital operations:")
    print("   - Average O(1) search supports quick decisions by doctors
and staff.")
    print("3. Sorting requirements:")
    print("   - O(n log n) sorting is practical for triage dashboards
and billing reports.")
    print("4. Update suitability:")
    print("   - Moderate patient updates are handled with index
rebuilds for clarity.")


def build_sample_system() -> PatientManagementSystem:
    patients = [
        Patient(2001, "Asha Rao", 5, date(2026, 3, 20), 98000.0),
        Patient(2002, "Vikram Singh", 3, date(2026, 3, 22), 34000.0),
        Patient(2003, "Neha Patel", 4, date(2026, 3, 21), 67000.0),
        Patient(2004, "Rohit Kumar", 2, date(2026, 3, 24), 19000.0),
        Patient(2005, "Neha Patel", 1, date(2026, 3, 24), 12000.0),
    ]
    return PatientManagementSystem(patients)


def run_demo() -> None:
    system = build_sample_system()

    print("\nSearch patient by ID = 2003")
```

```
    print(system.search_by_patient_id(2003))

    print("\nSearch patients by name = 'neha patel'")
    print(system.search_by_name("neha patel"))

    print("\nSort by severity (highest first)")
    print(system.sort_by_severity())

    print("\nSort by bill amount (highest first)")
    print(system.sort_by_bill_amount())


def run_tests() -> None:
    system = build_sample_system()

    # Search tests
    assert system.search_by_patient_id(2001) == Patient(
        2001, "Asha Rao", 5, date(2026, 3, 20), 98000.0
    )
    assert system.search_by_patient_id(9999) is None

    neha_records = system.search_by_name("Neha Patel")
    assert len(neha_records) == 2

    # Sort tests
    severity_ids = [patient.patient_id for patient in
system.sort_by_severity()]
    assert severity_ids == [2001, 2003, 2002, 2004, 2005]

    bill_ids = [patient.patient_id for patient in
system.sort_by_bill_amount()]
    assert bill_ids == [2001, 2003, 2002, 2004, 2005]

    print("\nAll test checks passed.")


def main() -> None:
    print("Smart Hospital Patient Management System")
    print_algorithm_table()
    print_justification_notes()
    run_demo()
    run_tests()
```

```python
if __name__== "__main__":
    main()
```

Output:
Smart Hospital Patient Management System
Operation | Recommended Algorithm | Justification

Search by patient ID | Hash index (dict) | Average O(1) lookup is ideal for frequent real-time access in large records.
Search by patient name | Normalized hash index (name -> list[Patient]) | Average O(1) lookup with case-insensitive matching; handles duplicate names.
Sort by severity level | Timsort (Python sorted) | Stable O(n log n) worst-case performance, optimized in Python for practical data.
Sort by bill amount | Timsort (Python sorted) | Efficient and stable for billing analysis and report generation on large datasets.

Suitability Notes
1. Efficiency for large datasets (thousands of records):
   - Hash indexes avoid linear scans during common search operations.
2. Real-time hospital operations:
   - Average O(1) search supports quick decisions by doctors and staff.
3. Sorting requirements:
   - O(n log n) sorting is practical for triage dashboards and billing reports.
4. Update suitability:
   - Moderate patient updates are handled with index rebuilds for clarity.

Search patient by ID = 2003

Patient(patient_id=2003, name='Neha Patel', severity_level=4,
admission_date=datetime.date(2026, 3, 21), bill_amount=67000.0)

Search patients by name = 'neha patel'
[Patient(patient_id=2003, name='Neha Patel', severity_level=4,
admission_date=datetime.date(2026, 3, 21), bill_amount=67000.0),
Patient(patient_id=2005, name='Neha Patel', severity_level=1,
admission_date=datetime.date(2026, 3, 24), bill_amount=12000.0)]

Sort by severity (highest first)
[Patient(patient_id=2001, name='Asha Rao', severity_level=5,
admission_date=datetime.date(2026, 3, 20), bill_amount=98000.0),
Patient(patient_id=2003, name='Neha Patel', severity_level=4,
admission_date=datetime.date(2026, 3, 21), bill_amount=67000.0),
Patient(patient_id=2002, name='Vikram Singh', severity_level=3,
admission_date=datetime.date(2026, 3, 22), bill_amount=34000.0),
Patient(patient_id=2004, name='Rohit Kumar', severity_level=2,
admission_date=datetime.date(2026, 3, 24), bill_amount=19000.0),
Patient(patient_id=2005, name='Neha Patel', severity_level=1,
admission_date=datetime.date(2026, 3, 24), bill_amount=12000.0)]

Sort by bill amount (highest first)
[Patient(patient_id=2001, name='Asha Rao', severity_level=5,
admission_date=datetime.date(2026, 3, 20), bill_amount=98000.0),
Patient(patient_id=2003, name='Neha Patel', severity_level=4,
admission_date=datetime.date(2026, 3, 21), bill_amount=67000.0),
Patient(patient_id=2002, name='Vikram Singh', severity_level=3,
admission_date=datetime.date(2026, 3, 22), bill_amount=34000.0),
Patient(patient_id=2004, name='Rohit Kumar', severity_level=2,
admission_date=datetime.date(2026, 3, 24), bill_amount=19000.0),
Patient(patient_id=2005, name='Neha Patel', severity_level=1,
admission_date=datetime.date(2026, 3, 24), bill_amount=12000.0)]

All test checks passed.

Justification:
Patient ID/name search uses dictionary-based indexing to support near-instant retrieval (average O(1)), which is critical for hospital workflows. Severity and bill sorting uses Timsort (O(nlogn)), suitable for prioritization and billing while maintaining stable and efficient ordering.

Task Description #5: University Examination Result Processing System
A university processes examination results for thousands of students
containing roll number, name, subject, and marks. The system must:
1. Search student results using roll numbers.
2. Sort students based on marks to generate rank lists.
Student Task
• Identify efficient searching and sorting algorithms using AI assistance.
• Justify the choice of algorithms.
• Implement the algorithms in Python.

Code:

```
"""Task #5: University Examination Result Processing System.

System requirements:
1. Search student results by roll number
2. Sort students by marks to generate rank lists

This implementation includes:
- Recommended algorithms with justifications
- Efficient Python search/sort functions
- Sample data and validation checks
"""

from dataclasses import dataclass
from typing import Dict, List
```

```python
@dataclass(frozen=True)
class StudentResult:
    roll_number: int
    name: str
    subject: str
    marks: int


ALGORITHM_TABLE = [
    {
        "operation": "Search by roll number",
        "algorithm": "Hash index (Python dict)",
        "justification": (
            "Average O(1) lookup is ideal for fast direct access among thousands of students."
        ),
    },
    {
        "operation": "Sort by marks (rank list)",
        "algorithm": "Timsort (Python sorted)",
        "justification": (
            "Stable O(n log n) sorting, highly optimized in CPython, suitable for large"
            " academic datasets and repeated ranking reports."
        ),
    },
]


class ResultProcessingSystem:
    def __init__(self, results: List[StudentResult]) -> None:
        self.results = list(results)
        self._roll_index: Dict[int, StudentResult] = {
            result.roll_number: result for result in self.results
        }

    def search_by_roll_number(self, roll_number: int) -> StudentResult | None:
        """Find a student result in average O(1) time."""
        return self._roll_index.get(roll_number)
```

```
    def sort_by_marks(self, descending: bool = True) ->
List[StudentResult]:
        """Generate rank list. Default descending: highest marks
first."""
        return sorted(self.results, key=lambda result: result.marks,
reverse=descending)


def print_algorithm_table() -> None:
    print("Operation | Recommended Algorithm | Justification")
    print("-" * 110)
    for row in ALGORITHM_TABLE:
        print(f"{row['operation']} | {row['algorithm']} |
{row['justification']}")


def print_justification_notes() -> None:
    print("\nWhy these algorithms are suitable")
    print("1. Dataset size (thousands of records):")
    print("   - Dictionary indexing avoids O(n) linear search for roll-
number queries.")
    print("2. Performance need (quick student lookup):")
    print("   - O(1) average search supports instant result
retrieval.")
    print("3. Ranking requirement:")
    print("   - O(n log n) Timsort is efficient and stable for marks-
based rank lists.")


def build_sample_system() -> ResultProcessingSystem:
    sample_results = [
        StudentResult(501, "Anita", "Mathematics", 92),
        StudentResult(502, "Rahul", "Mathematics", 81),
        StudentResult(503, "Priya", "Mathematics", 95),
        StudentResult(504, "Karan", "Mathematics", 88),
        StudentResult(505, "Sneha", "Mathematics", 95),
    ]
    return ResultProcessingSystem(sample_results)


def run_demo() -> None:
    system = build_sample_system()
```

```
    print("\nSearch by roll number = 503")
    print(system.search_by_roll_number(503))

    print("\nRank list (marks descending)")
    print(system.sort_by_marks())


def run_tests() -> None:
    system = build_sample_system()

    # Search tests
    assert system.search_by_roll_number(501) == StudentResult(
        501, "Anita", "Mathematics", 92
    )
    assert system.search_by_roll_number(999) is None

    # Sorting tests
    ranked = system.sort_by_marks()
    ranked_rolls = [item.roll_number for item in ranked]
    assert ranked_rolls == [503, 505, 501, 504, 502]

    # Ascending check
    ascending_rolls = [item.roll_number for item in
system.sort_by_marks(descending=False)]
    assert ascending_rolls == [502, 504, 501, 503, 505]

    print("\nAll validation checks passed.")


def main() -> None:
    print("University Examination Result Processing System")
    print_algorithm_table()
    print_justification_notes()
    run_demo()
    run_tests()


if __name__ == "__main__":
    main()
```

Output:
University Examination Result Processing System
Operation | Recommended Algorithm | Justification

Search by roll number | Hash index (Python dict) | Average O(1) lookup is ideal for fast direct access among thousands of students.
Sort by marks (rank list) | Timsort (Python sorted) | Stable O(n log n) sorting, highly optimized in CPython, suitable for large academic datasets and repeated ranking reports.

Why these algorithms are suitable
1. Dataset size (thousands of records):
   - Dictionary indexing avoids O(n) linear search for roll-number queries.
2. Performance need (quick student lookup):
   - O(1) average search supports instant result retrieval.
3. Ranking requirement:
   - O(n log n) Timsort is efficient and stable for marks-based rank lists.

Search by roll number = 503
StudentResult(roll_number=503, name='Priya', subject='Mathematics', marks=95)

Rank list (marks descending)
[StudentResult(roll_number=503, name='Priya', subject='Mathematics', marks=95), StudentResult(roll_number=505, name='Sneha', subject='Mathematics', marks=95), StudentResult(roll_number=501, name='Anita', subject='Mathematics', marks=92), StudentResult(roll_number=504, name='Karan', subject='Mathematics', marks=88), StudentResult(roll_number=502, name='Rahul', subject='Mathematics', marks=81)]

All validation checks passed.

Justification:
Roll-number search uses a hash map for average O(1) retrieval, which scales well for thousands of student records.
Marks-based rank generation uses Timsort (O(nlogn)), providing efficient and stable sorting for result publishing.

Task Description #6: Online Food Delivery Platform
An online food delivery application stores thousands of orders with order ID, restaurant name, delivery time, price, and order status. The
platform needs to:
1. Quickly find an order using order ID.
2. Sort orders based on delivery time or price.
Student Task
• Use AI to suggest optimized algorithms.
• Justify the algorithm selection.
• Implement searching and sorting modules in Python.

Code:

```
"""Task #6: Online Food Delivery Platform.

Requirements:
1. Quickly find an order using order ID.
2. Sort orders by delivery time or price.

This script includes:
- Operation -> algorithm -> justification table
- Efficient searching and sorting modules in Python
- Sample data and simple validation checks
"""

from dataclasses import dataclass
from typing import Dict, List


@dataclass(frozen=True)
class Order:
    order_id: int
    restaurant_name: str
    delivery_time: int  # in minutes
    price: float
    order_status: str


ALGORITHM_TABLE = [
    {
        "operation": "Search by order ID",
        "algorithm": "Hash index (Python dict)",
        "justification": (
            "Average O(1) lookup is ideal for real-time order tracking
in large systems."
        ),
    },
    {
        "operation": "Sort by delivery time",
        "algorithm": "Timsort (Python sorted)",
        "justification": (
            "O(n log n) worst-case, stable, and optimized in CPython
for practical workloads."
        ),
    },
```

```
    {
        "operation": "Sort by price",
        "algorithm": "Timsort (Python sorted)",
        "justification": (
            "Efficient and stable for analytics, pricing views, and
repeated report generation."
        ),
    },
]


class FoodDeliverySystem:
    def __init__(self, orders: List[Order]) -> None:
        self.orders = list(orders)
        self._id_index: Dict[int, Order] = {order.order_id: order for
order in self.orders}

    def search_by_order_id(self, order_id: int) -> Order | None:
        """Find one order in average O(1) time using dictionary
indexing."""
        return self._id_index.get(order_id)

    def sort_by_delivery_time(self, descending: bool = False) ->
List[Order]:
        """Sort orders by delivery time. Default ascending for fastest
deliveries first."""
        return sorted(self.orders, key=lambda order:
order.delivery_time, reverse=descending)

    def sort_by_price(self, descending: bool = False) -> List[Order]:
        """Sort orders by total price. Default ascending for lower-
price-first view."""
        return sorted(self.orders, key=lambda order: order.price,
reverse=descending)


def print_algorithm_table() -> None:
    print("Operation | Recommended Algorithm | Justification")
    print("-" * 112)
    for row in ALGORITHM_TABLE:
        print(f"{row['operation']} | {row['algorithm']} |
{row['justification']}")
```

```
def print_justification_notes() -> None:
    print("\nWhy these algorithms are suitable")
    print("1. Dataset size (thousands of orders):")
    print("   - Dictionary indexing prevents repeated O(n) scans for
order lookup.")
    print("2. Real-time requirement:")
    print("   - O(1) average ID search provides near-instant order
tracking.")
    print("3. Sorting requirement:")
    print("   - O(n log n) sorting is practical for delivery planning
and pricing analysis.")
    print("4. Update pattern:")
    print("   - For frequent reads and moderate writes, hash indexes
are a strong fit.")


def build_sample_system() -> FoodDeliverySystem:
    sample_orders = [
        Order(3001, "Spice Hub", 28, 449.0, "Out for Delivery"),
        Order(3002, "Urban Bites", 42, 799.0, "Preparing"),
        Order(3003, "Green Bowl", 20, 299.0, "Delivered"),
        Order(3004, "Pizza Planet", 35, 599.0, "Out for Delivery"),
        Order(3005, "Spice Hub", 25, 349.0, "Confirmed"),
    ]
    return FoodDeliverySystem(sample_orders)


def run_demo() -> None:
    system = build_sample_system()

    print("\nSearch order by ID = 3004")
    print(system.search_by_order_id(3004))

    print("\nSort by delivery time (ascending)")
    print(system.sort_by_delivery_time())

    print("\nSort by price (descending)")
    print(system.sort_by_price(descending=True))


def run_checks() -> None:
    system = build_sample_system()
```

```
    # Search checks
    assert system.search_by_order_id(3001) == Order(
        3001, "Spice Hub", 28, 449.0, "Out for Delivery"
    )
    assert system.search_by_order_id(9999) is None

    # Sort checks
    by_time_ids = [order.order_id for order in
system.sort_by_delivery_time()]
    assert by_time_ids == [3003, 3005, 3001, 3004, 3002]

    by_price_desc_ids = [order.order_id for order in
system.sort_by_price(descending=True)]
    assert by_price_desc_ids == [3002, 3004, 3001, 3005, 3003]

    print("\nAll validation checks passed.")


def main() -> None:
    print("Online Food Delivery Platform - Search and Sort")
    print_algorithm_table()
    print_justification_notes()
    run_demo()
    run_checks()


if __name__ == "__main__":
    main()
```

Output:
Online Food Delivery Platform - Search and Sort
Operation | Recommended Algorithm | Justification

Search by order ID | Hash index (Python dict) | Average O(1) lookup is ideal for real-time order tracking in large systems.
Sort by delivery time | Timsort (Python sorted) | O(n log n) worst-case, stable, and optimized in CPython for practical workloads.

Sort by price | Timsort (Python sorted) | Efficient and stable for analytics, pricing views, and repeated report generation.

Why these algorithms are suitable
1. Dataset size (thousands of orders):
   - Dictionary indexing prevents repeated O(n) scans for order lookup.
2. Real-time requirement:
   - O(1) average ID search provides near-instant order tracking.
3. Sorting requirement:
   - O(n log n) sorting is practical for delivery planning and pricing analysis.
4. Update pattern:
   - For frequent reads and moderate writes, hash indexes are a strong fit.

Search order by ID = 3004
Order(order_id=3004, restaurant_name='Pizza Planet', delivery_time=35, price=599.0, order_status='Out for Delivery')

Sort by delivery time (ascending)
[Order(order_id=3003, restaurant_name='Green Bowl', delivery_time=20, price=299.0, order_status='Delivered'), Order(order_id=3005, restaurant_name='Spice Hub', delivery_time=25, price=349.0, order_status='Confirmed'), Order(order_id=3001, restaurant_name='Spice Hub', delivery_time=28, price=449.0, order_status='Out for Delivery'), Order(order_id=3004, restaurant_name='Pizza Planet', delivery_time=35, price=599.0, order_status='Out for Delivery'), Order(order_id=3002, restaurant_name='Urban Bites', delivery_time=42, price=799.0, order_status='Preparing')]

Sort by price (descending)
[Order(order_id=3002, restaurant_name='Urban Bites', delivery_time=42, price=799.0, order_status='Preparing'),

Order(order_id=3004, restaurant_name='Pizza Planet', delivery_time=35, price=599.0, order_status='Out for Delivery'), Order(order_id=3001, restaurant_name='Spice Hub', delivery_time=28, price=449.0, order_status='Out for Delivery'), Order(order_id=3005, restaurant_name='Spice Hub', delivery_time=25, price=349.0, order_status='Confirmed'), Order(order_id=3003, restaurant_name='Green Bowl', delivery_time=20, price=299.0, order_status='Delivered')]

All validation checks passed.

Justification:
Order tracking by order ID uses dictionary lookup (average O(1)), enabling fast response in high-volume order systems.
Sorting by delivery time/price uses Timsort (O(nlogn)), which is effective for logistics prioritization and pricing analysis.